REACT

React is an open-source JavaScript library used for building user interfaces (UIs) — especially single-page applications (SPAs) where data changes dynamically without reloading the entire page.

Facebook (Jordan Walke) introduced React in the year 2011 and made it as an open source in 2013.

Feature	**Single Page Application (SPA)**	**Multi Page Application (MPA)**
Definition	The whole app runs on a single HTML page. Only specific parts of the page are updated dynamically.	Each page is a separate HTML file, and the browser reloads the page when you navigate.
Page Reload	No full page reload — only part of the page updates.	Every navigation causes a full page reload.
Speed	Faster	Slower
Technology Example	React, Angular, Vue	Traditional apps like PHP, ASP.NET, WordPress
User Experience	Smooth and app-like (no page flicker).	Feels like navigating between pages (with reloads).
SEO (Search Engine Optimization)	Harder to optimize (since content loads dynamically).	Easier to optimize (each page has its own content).
Development Complexity	Easier to build and maintain for small/medium apps.	More suitable for large apps with many distinct pages.
Example	Gmail, Facebook, Twitter	Amazon, Wikipedia, news websites

Features of react

1] Declarative

You describe what you want the UI to look like, and React takes care of how to make it happen. React automatically updates the UI when data changes.

2] Component Based Architecture

Everything in React is built using components.

A component is a reusable, independent piece of UI (like a button, card, or navbar).

Components can be functional or class-based.

3] Unidirectional data flow

React data flows only in one direction (top to bottom). This makes the app predictable and easier to debug.

4] Learn once, write anywhere

5]Virtual DOM

React maintains a virtual copy of the real DOM in memory.

When data changes, React compares the old and new Virtual DOM and updates only the changed parts in the actual DOM. This improves performance and speed.

Virtual DOM

Virtual DOM is a copy of real DOM, you can say a xerox copy.

Whatever updates are made is not reflected on the real dom or the screen directly.

It does all the manipulation within itself and then render those changes on the real dom so in this way we do not need to update the real dom again and again.

Diffing :

React compares the virtual copy of the real dom to an updated copy of virtual dom , compares or picks out the changes. This process is called as diffing and the algorithm used is called as diffing algorithm.

Reconciliation:

When a component(node) changes , react decides whether it should render the changes on real dom or not.

So if the states/props of two nodes/components are not same, then it renders the changes to Real DOM.

This process is called as Reconciliation.

In react, when the state of a component changes, the component needs to update its UI to reflect new state.

This process of updating the UI is called as reconciliation. It is dependent on :

Virtual DOM

Diffing Algorithm

Installation of vite

1] Install node JS (after installing close everything and install it only once)

2]Create a folder

3] Open with cmd (Command Prompt)

4] We need to pass a command: npm create vite@latest

5]Ok to proceed? yes

6] project name: project-name

7] Select the framework: React

8] Select variant: JavaScript

9] cd project-name

10] npm install

11] open with vs code

12] Run the code : npm run dev

Why React JS?

To create Single page applications.

It is Declarative

It follows Component Based Architecture.

Folder Structure:

node-modules:

All the pre-defined codes are present in this folder (do not touch it)

src :

It is a source folder where we are going to write the code. 2] Inside src you have two important files i.e

main.jsx :This will create a connection between the src and index.html

App.jsx: It is a component where we will be writing our code.

package-lock.json & package.json: These are considered as directories of the react folder.It will provide you information about libraries present in the project.

JSX

JavaScript Extensible Markup Language It is a syntax extension for JavaScript.
It is a template language of React.

Rules of JSX:

1] JSX must be closed, which means each and every element should be closed.

<h1>some content</h1>

Even if it is a void tag or empty tag then close the element there itself.

<input/>

2] JSX must have one parent element or else you can get one error. Adjacent elements should be wrapped in an enclosing tag.

<h1> some content </h1>

<h2> some content </h2>

3] The tag names should always be in lowercase.

<h1> some content </h1>

<H1> some content </H1> Throws an error

4] In html we used attribute 'class' and 'for'. But in jsx we will use 'className' and 'htmlFor'.

5]All the events we use should be in lowerCamel case. Ex: onClick, onSubmit, onMouseOver

JSX Expressions:

With JSX you can write expressions inside curly braces {}.

The expression can be a react variable, or property or any other valid javaScript expression.

JSX will execute the expression and return the result.

COMPONENTS:

Components are nothing but building blocks of the web page.

Webpage will be divided into multiple components(files) and then we will be joining together in the parent component (App.jsx).

Components are reusable.

Rules for creating a component:

First letter of the file name should always be capital.

File extension of component is .jsx

We can declare components in two ways:

1]Self-closing tag (<ComponentName/>)

2] Paired tag (<ComponentName></ComponentName>)

Types of Components:

Functional Based Component

Class Based Component

1] Functional Based Component:

It uses javaScript function.

FBC is stateless.

We can use Hooks.

Can't use life-cycle methods.

render() is not used.

```
const Func = ()=>{ return (
<h1>Hello</h1>
)
}
```

2] Class Based Component:

It uses class.

CBC is statefull.

We cannot use Hooks.

We use life-cycle methods.

render() is used.

```
import {Component} from 'react'

class CcomponentName extends Component {

render(){

return(

<h1> hello </h1> )

}

}
```

FRAGMENT

Fragment let you group a list of children without adding extra node to the DOM.

Fragment can accept only "key" prop as a property.

Except this it will not accept any other properties.

1st way

```
import React from 'react'

<React.Fragment>

<h1>You can give only one prop i.e key</h1>

<h2>Functional Based Component</h2>

</React.Fragment>
```

2nd way

```
import {Fragment} from 'react'

<Fragment>

<h1>You can give only one prop i.e key</h1>

<h2>Functional Based Component</h2>

</Fragment>
```

3rd way

```
<>

<h1>You cannot give any props</h1>

<h2>Functional Based Component</h2>

</>
```

** key prop provides unique identity for each data(value) inside the array which you are mapping.